

Reporte de la práctica

Semáforo vehicular y peatonal con máquina de estados finitos (FSM)

1. Datos generales

Materia	Sistemas Programables
Institución	Instituto Tecnológico de Mazatlán (TecNM)
Alumno	Jesús Antonio Rodríguez Peraza — L231000105
Práctica	Práctica 3 — Semáforo vehicular y peatonal con máquina de estados finitos (FSM)
Placa utilizada	Arduino UNO R4 WiFi
Componentes	Arduino UNO R4 WiFi, 5 LEDs (rojo, amarillo y verde vehicular; rojo y verde peatonal), 5 resistencias de 220 Ω , pulsador normalmente abierto, protoboard, cables Dupont
Programa	SemaforoFSM.ino (Codigo/SemaforoFSM/): enum class Estado, switch de transiciones, sin delay()
Tiempos	Verde 6 s, amarillo 2 s, rojo 5 s, cruce peatonal 5 s, antirrebote 40 ms
Repositorio	github.com/Jesus-Jazisenot/Universidad → Sistemas-Programables/Practica-3-Semaforo-Vehicular-y-Peatonal-FSM

2. Objetivo

Implementar un semáforo vehicular de tres luces que cicle solo, más un semáforo peatonal de dos luces con pulsador de cruce, modelando el problema como una máquina de estados finitos: estados en un enum class, transiciones por tiempo y por evento en un switch, temporización 100 % con millis() y una petición peatonal que se arma con el botón y se atiende solo cuando el vehicular llega a rojo.

3. Diagrama del circuito (conexiones)

Elemento	Pin Arduino	Nota
Vehicular VERDE	11	220 Ω a GND
Vehicular AMARILLO	12	220 Ω a GND
Vehicular ROJO	13	220 Ω a GND
Peatonal ROJO	8	220 Ω a GND
Peatonal VERDE	7	220 Ω a GND
Pulsador de cruce	2	Otra pata a GND; INPUT_PULLUP (presionado = LOW), sin resistencia externa

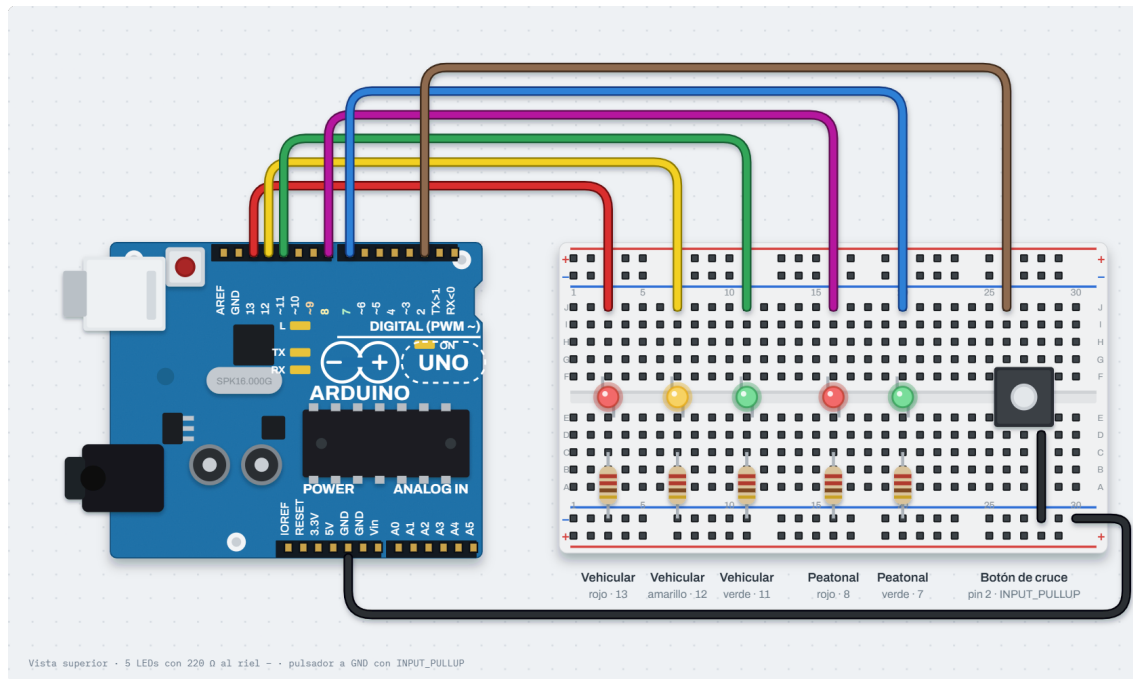


Figura 1. Diagrama de conexiones del circuito.

4. Procedimiento

1. Se definieron los cuatro estados (VEH_VERDE, VEH_AMARILLO, VEH_ROJO, PEA_VERDE) y sus transiciones en un diagrama de estados antes de escribir código.
2. Se armó el circuito: cinco LEDs con resistencia de $220\ \Omega$ en los pines 11, 12, 13 (vehicular) y 8, 7 (peatonal); el pulsador del pin 2 a GND, usando INPUT_PULLUP.
3. Se escribió SemaforoFSM.ino con un enum class Estado, una función aplicarSalidas() que escribe las cinco luces según el estado, y un switch en el loop() que evalúa las transiciones por tiempo con millis().
4. Se implementó la lectura del pulsador con antirrebote no bloqueante (40 ms); la pulsación solo arma solicitudPeatonal en VERDE o AMARILLO, y se ignora en los demás estados.
5. Se cargó el programa, se abrió el Monitor Serie a 9600 baudios y se hicieron las pruebas: ciclo libre, pulsar en verde, en amarillo, en rojo y durante el cruce peatonal (tabla de la sección 5).
6. Se grabó el video del funcionamiento y se verificaron los tiempos con los instantes del serial.

5. Tabla de estados y pruebas realizadas

Tabla de estados de la FSM, con lo que enciende cada uno y su transición de salida:

Estado	Vehicular	Peatonal	Duración	Sale hacia
VEH_VERDE	verde	rojo	6 s	VEH_AMARILLO
VEH_AMARILLO	amarillo	rojo	2 s	PEA_VERDE si hay solicitud armada; si no, VEH_ROJO
VEH_ROJO	rojo	rojo	5 s	VEH_VERDE
PEA_VERDE	rojo	verde	5 s	VEH_VERDE

Pruebas realizadas y comportamiento observado:

Prueba	Qué se hizo	Resultado observado
Ciclo libre	No tocar el botón	VERDE 6 s → AMARILLO 2 s → ROJO 5 s → VERDE; el peatonal siempre en rojo
Botón en VERDE	Pulsar durante el verde vehicular	Se imprime "solicitud ARMADA"; el verde termina completo, pasa por amarillo y en vez de VEH_ROJO entra PEA_VERDE 5 s
Botón en AMARILLO	Pulsar durante el amarillo	Igual que en verde: la solicitud se arma y se atiende al llegar a rojo
Botón en ROJO	Pulsar con el vehicular en rojo	Se imprime "boton ignorado"; nada cambia y el ciclo continúa
Botón en PEA_VERDE	Pulsar con el peatonal en verde	Se ignora: no se alarga el cruce ni se encadena otro
Doble pulsación	Pulsar dos veces en el mismo verde	Solo se arma una solicitud; la segunda no genera mensaje
Antirrebote	Pulsaciones cortas o "sucias"	Una sola solicitud por pulsación; sin dobles disparos

* Los tiempos se verificaron con los instantes que imprime el Monitor Serie en cada transición (0, 6000, 8000, 13000 ms...), que corresponden a las constantes T_VERDE, T_AMARILLO, T_ROJO y T_PEATON del programa.

6. Salida del Monitor Serie (9600 baudios)

```
Semaforo FSM (sin delay)
0 ms -> VEH_VERDE
3210 ms boton: solicitud peatonal ARMADA, se atiende en rojo
6000 ms -> VEH_AMARILLO (solicitud peatonal armada)
8000 ms -> PEA_VERDE
13000 ms -> VEH_VERDE
19000 ms -> VEH_AMARILLO
21000 ms -> VEH_ROJO
22500 ms boton ignorado (fuera de verde/amarillo)
26000 ms -> VEH_VERDE
```

7. Observaciones sobre el comportamiento del sistema

- El ciclo vehicular VERDE (6 s) → AMARILLO (2 s) → ROJO (5 s) se repite solo, sin necesidad de pulsar nada, y el peatonal permanece en rojo mientras tanto.
- Al pulsar en verde o amarillo el cambio no es inmediato: el vehicular completa su fase y, al llegar a rojo, el peatonal pasa a verde 5 s. Luego ambos regresan al ciclo normal.
- Pulsar en rojo o durante el cruce peatonal no tiene efecto (se imprime "boton ignorado"); no se encadenan cruces ni se alarga el rojo vehicular.
- Nunca se observaron los dos verdes encendidos a la vez: aplicarSalidas() escribe las cinco luces a partir del estado, por lo que las salidas no pueden quedar en una combinación inválida.
- El antirrebote de 40 ms sin delay() eliminó los dobles disparos del pulsador; una pulsación genera exactamente una solicitud.
- Los instantes impresos en el Monitor Serie coinciden con las constantes de tiempo del programa, lo que confirma que millis() lleva la temporización sin acumular desfase.

8. Conclusiones

Pensar el semáforo como máquina de estados antes de programarlo simplifica todo: cada estado dice qué luces van encendidas y qué lo saca de ahí, y el código termina siendo la tabla de transiciones escrita como un switch. Agregar un estado (por ejemplo un parpadeo del peatonal antes de terminar) es agregar un renglón a esa tabla, no reescribir el programa.

enum class en lugar de enum clásico evita errores concretos: no se puede comparar el estado contra un número suelto ni contra otro enum sin que el compilador lo rechace, y los nombres quedan encapsulados (Estado::VEH_ROJO). Un error que con enum pasaría en silencio aquí no compila.

La solicitud "armada" es la pieza clave: el botón no cambia de estado en el instante en que se pulsa, solo levanta una bandera que se atiende en el momento seguro (al terminar el amarillo). Así el vehicular nunca pasa de verde a rojo de golpe y es imposible que los dos semáforos estén en verde al mismo tiempo, porque las cinco luces las escribe una sola función a partir del estado.

Todo con millis() permite leer el botón en cada vuelta del loop(); con delay() la pulsación se perdería mientras el programa espera los 6 s del verde. El antirrebote también es no bloqueante, por lo que el semáforo nunca deja de atender el tiempo.

Se probó pulsar en verde, en amarillo, en rojo y no pulsar. En los dos primeros casos se atiende el cruce al llegar a rojo; en rojo se ignora; sin pulsar, el ciclo sigue solo. El Monitor Serie con el instante de cada transición fue suficiente para comprobar los tiempos sin cronómetro.

9. Evidencia del armado

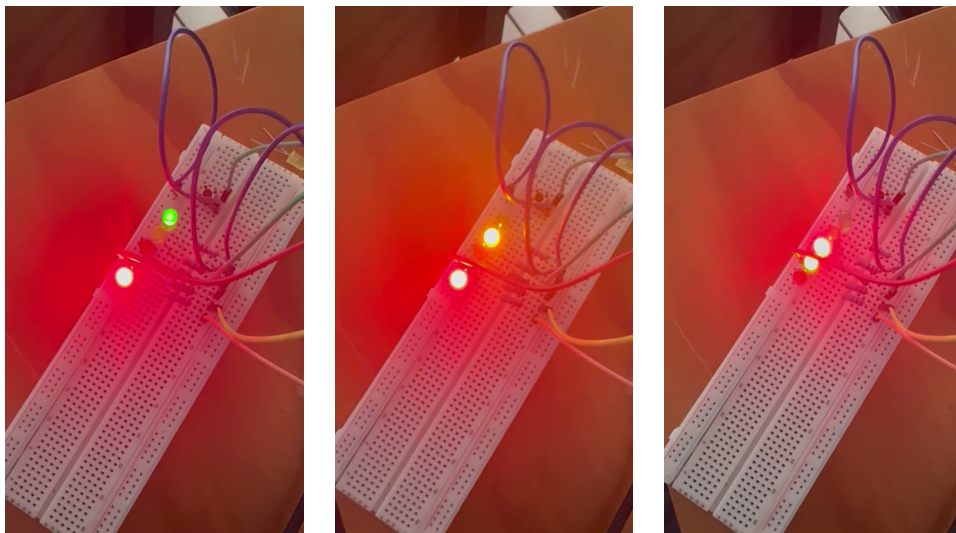


Figura 2. Circuito armado, ciclo vehicular y el instante con ambos semáforos en rojo (cuadros del video).

Video del funcionamiento: <https://youtube.com/shorts/jQlzcY6zho>